Word2vec Parameter Learning Explained

Jeong Hoon Choi
USCID: 5023184813

DSCI-560
Data Science Professional Practicum

Continuous Bag-of-Word Model

one-hot encoding

$$X_{(1\times V)} \cdot W_{(V\times N)} = h_{(1\times N)}$$

$$h_{(1\times N)} \cdot W_{(N\times V)} = y_{(1\times V)}$$

$$u_j = v'^{T}_{w_j} h$$

$$p(w_j|w_I) = y_j = \frac{\exp(u_j)}{\sum_{j'=1}^{V} \exp(u_{j'})}$$

Hidden -> Output Weights update

Objective Function

$$\max p(w_o|w_I) = \max y_j^*$$

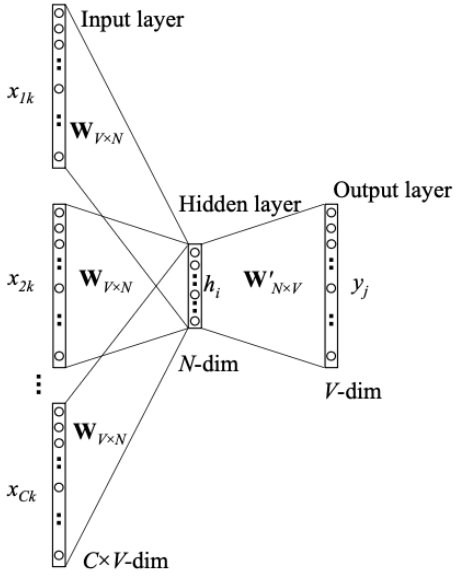
$$= \max \log(y_j^*)$$

$$= u_j^* - \log \sum \exp(u_j) := -E$$

$$\Rightarrow -\frac{\partial}{\partial u_j}\left(\log \sum \exp(u_j) - u_j^*\right) = y_j - t_j = e_j$$

$$\therefore \frac{\partial E}{\partial w'_{ij}} = \frac{\partial E}{\partial u_j} \cdot \frac{\partial u_j}{\partial w'_{ij}} = e_j \cdot h_i$$

Continuous Bag-of-Word Model



Input -> Hidden Weights update

$$\frac{\partial E}{\partial h_i} = \sum_{j=1}^{V} \frac{\partial E}{\partial u_j} \cdot \frac{\partial u_j}{\partial h_i} = \sum_j e_j \cdot w'_{ij} = EH_i$$

$$\Rightarrow \frac{\partial E}{\partial w_{ki}} = \frac{\partial E}{\partial h_i} \cdot \frac{\partial h_i}{\partial w_{ki}} = EH_i \cdot x_k$$

$$\because \; h_i = \sum_{k=1}^{V} x_k \cdot w_{ki}$$

From Single-word Context to Multi-word Context

As with Single Word Context, for C Word Contexts, the input vector (One-Hot Encoding) can be thought of as C independent W_(V x N) weights and one W'_(N x V). Average vector as the output (hidden)

CBOW predicts the middle word for surrounding words (CONTEXT). In one-word and multi-word contexts, these correspond to the input. A model is created that predicts the middle word for the CONTEXT (one-hot encoding) and predicts the label.

Skip-Gram Model

$$p(w_{c,j} = w_{o,c} \mid w_I) = y_{c,j} = \frac{\exp(u_{c,j})}{\sum_{j'} \exp(u_{j'})}$$

$$\Rightarrow E = -\log p(w_{o,1}, w_{o,2}, \cdots, w_{o,c} \mid w_I)$$

$$= -\log \prod_{c=1}^{C} p(w_{o,c} \mid w_I) = -\log \prod_{c=1}^{C} \frac{\exp(u_{c,j})}{\sum_{j'} \exp(u_{j'})}$$

$$= -\sum_{c=1}^{C} u_{j_c^*} + C \cdot \log \sum_{j'} \exp(u_{j'})$$

Hidden -> Output Weights update

$$\frac{\partial E}{\partial w'_{ij}} = \sum_{c=1}^{C} \frac{\partial E}{\partial u_{c,j}} \cdot \frac{\partial u_{c,j}}{\partial w'_{ij}} = E I_j \cdot h_i$$

Input -> Hidden Weights update

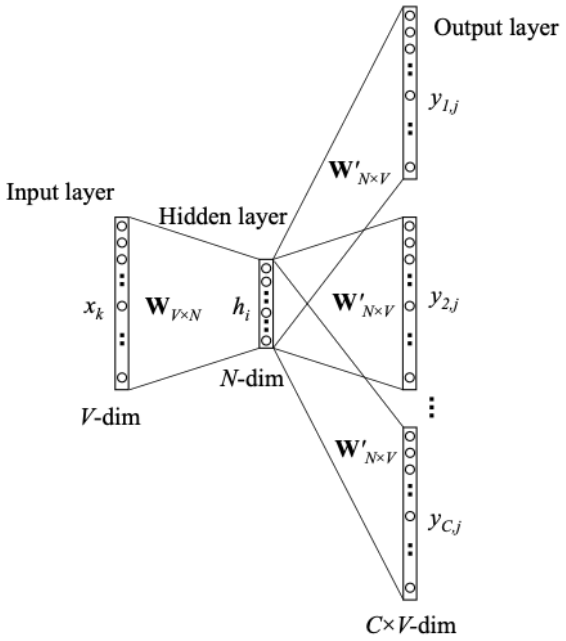
Same with CBOW

$$\frac{\partial E}{\partial w_{ki}} = \frac{\partial E}{\partial h_i} \cdot \frac{\partial h_i}{\partial w_{ki}} = E H_i \cdot X_k$$

Skip-Gram Model

Conversely, the Skip-Gram model predicts the surrounding word CONTEXT for a given target word.

The input is the middle word (one-hot encoding), and the output is a one-hot encoding of the words corresponding to the CONTEXT.



Optimizing Computational Efficiency

Hierarchical Softmax

Binary tree: all words in the leaf (in complete binary tree if leaf has V nodes total nodes are (2 V - 1)

For every words has unique path from the root to the unit

Instead output vector representation for words inner units has an output vector

Probability of word is conditional probability of each node on the path

O(V) -> O(log V): height of binary tree

Optimizing Computational Efficiency

Negative Sampling

Instead of updating the weights for all words, we only perform weight updates for their samples.

Samples for the correct label (TARGET for CBOW, CONTEXT for Skip-Gram) must be included, but for other labels (Negative), random samples are sampled and training is performed.

O(V) -> O(K): K is negative sampling size

Question:

Numerically represent and explain the degree of performance improvement of the Word2Vec optimization methods described in the paper.

Answer:

Naïve CBOW or Skip-Gram Word2Vec:

The biggest computational bottleneck occurs when calculating the MLE term. For example, when calculating the term we need to calculate whole word probability given CONTEXT.

Hierarchical Softmax Optimization:

The root of a word is represented in binary tree format, and only the vectors of the nodes on the path are updated. Therefore, the operator of O(V) can be reduced to O(log V) because it is calculated only with the node vectors on the path.

Negative Sampling Optimization:

When calculating the loss, instead of calculating it for every word, we sample K negative terms that include the target and calculate it. This reduces the computational complexity from the existing O(V) to O(K), where V >> K.